Independent University, Bangladesh

CSE313 Compiler Construction · Sample Midterm + Quiz (Practice) · Time: 1h30m

Same shape as Mid Autumn-23. New questions.

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following grammar is Ambiguous or NOT. If the grammar is ambiguous, convert it to unambiguous form. **[7]**

```
E → E - E | E / E | E ^ E | ( E ) | id
```

2. Write the First() and Follow() of the following grammar. Draw the LL(1) Parsing Table and parse the string `w x x y k z` showing all steps. If parsing is not possible, point out the reasons. (b) Using the same table, parse `x y w z`. **[8]**

```
S → X Y z | w S
X → x X | ε
Y → y Z | ε
Z → k | ε
```

3. Find True/False: **[5]**

(a) An LL(1) grammar can be left recursive.
(b) The FIRST set of a non-terminal can contain ε.
(c) A grammar with a FIRST/FIRST conflict can be parsed by an LL(1) parser.
(d) The lexical analyzer removes comments and white space.
(e) Left factoring changes the language of the grammar.

Quiz Question (Total marks: 10)

1. Convert the following Non-Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
S → a b c S | a b d | a b | a T | e
T → t
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
A → A b c | A d | e | f B
B → B g | h
```

Extra practice [8]

4. The grammar below is left recursive. Make it ready for LL(1), then write First() and Follow(), draw the LL(1) table and parse `int id ; float id`. **[8]**

```
L → L ; D | D
D → int id | float id
```

Answers (exam layout, with comments)

Answer 1 [7]

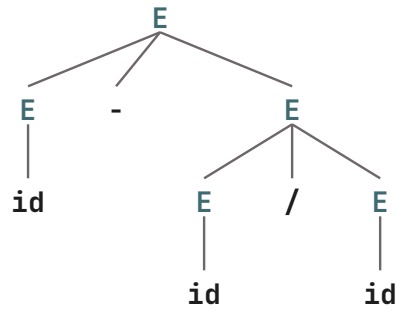
WRITE THIS

The grammar is ambiguous. String: `id - id / id`

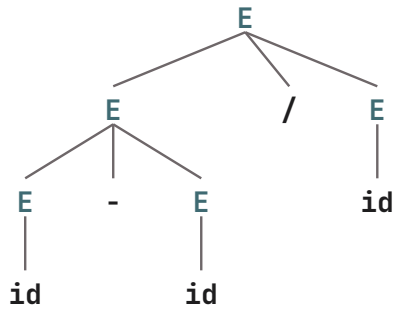
LMD 1 (- at the root: does / first, correct): E ⇒ E - E ⇒ id - E ⇒ id - E / E ⇒ id - id / E ⇒ id - id / id

LMD 2 (/ at the root: does - first, wrong): E ⇒ E / E ⇒ E - E / E ⇒ id - E / E ⇒ id - id / E ⇒ id - id / id

Tree 1



Tree 2



Since for the string `id - id / id` there exists more than one leftmost derivation parse tree, the grammar is ambiguous.

Operator	Precedence	Associativity
^	highest	right
/	middle	left
-	lowest	left

Unambiguous grammar (low precedence on top, high at the bottom):

```
E → E - T | T
T → T / F | F
F → G ^ F | G
G → ( E ) | id
```

- and / are left associative → left recursive. ^ is right associative → right recursive (F → G ^ F).

> **Lose marks by:** two trees that are really the same tree; forgetting (E); no assumption written.

Answer 2 [8]

WRITE THIS

Numbered rules: 1. S → X Y z 2. S → w S 3. X → x X 4. X → ε 5. Y → y Z 6. Y → ε 7. Z → k 8. Z → ε

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
S	{ w, x, y, z }	{ $ }
X	{ x, ε }	{ y, z }
Y	{ y, ε }	{ z }
Z	{ k, ε }	{ z }

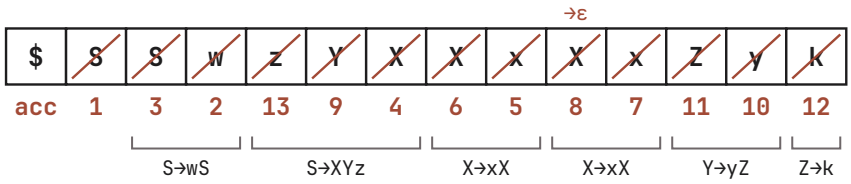
Step 2 · LL(1) parsing table

	z	w	x	y	k	$
S	S→XYz	S→wS	S→XYz	S→XYz		
X	X→ε		X→xX	X→ε		
Y	Y→ε			Y→yZ		
Z	Z→ε				Z→k	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3a · Parse `w x x y k z` (horizontal stack)

Buffer: `w x x y k z $` LA ↑ starts at `w`



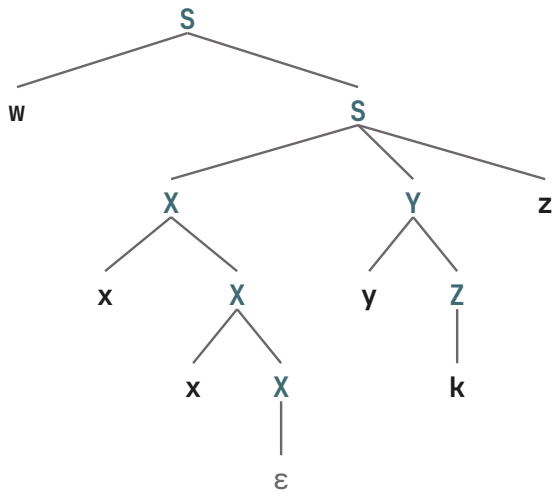
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop S (LA = w): S→wS, push S w
2. match w, pop it, move LA → x
3. pop S (LA = x): S→XYz, push z Y X
4. pop X (LA = x): X→xX, push X x
5. match x, pop it, move LA → x
6. pop X (LA = x): X→xX, push X x
7. match x, pop it, move LA → y
8. pop X (LA = y): X→ε, push nothing
9. pop Y (LA = y): Y→yZ, push Z y
10. match y, pop it, move LA → k
11. pop Z (LA = k): Z→k, push k
12. match k, pop it, move LA → z
13. match z, pop it, move LA → $

acc. Top = $ and LA = $ → accept

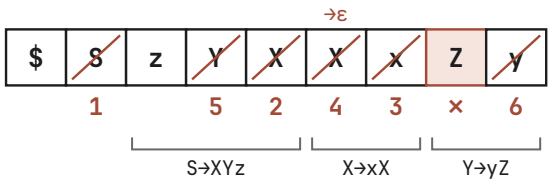
Parse tree



Leaves left to right (skip ε): `w x x y k z` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

Step 3b · Parse `x y w z` (horizontal stack)

Buffer: `x y w z $` LA ↑ starts at `x`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop S (LA = x): S→XYz, push z Y X
2. pop X (LA = x): X→xX, push X x
3. match x, pop it, move LA → y
4. pop X (LA = y): X→ε, push nothing
5. pop Y (LA = y): Y→yZ, push Z y
6. match y, pop it, move LA → w

err. Top = Z, LA = w: cell M[Z, w] is empty → error

Parsing is not possible. Reason: Top = Z, LA = w: cell M[Z, w] is empty → error. ∴ error, the string `x y w z` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(S): S → X Y z gives { x, y, z } (X can be ε → look at the next symbol; Y can be ε → look at the next symbol). S → w S gives { w }. Total { w, x, y, z }.
- FIRST(X): X → x X gives { x }. X → ε gives ε. Total { x, ε }.
- FIRST(Y): Y → y Z gives { y }. Y → ε gives ε. Total { y, ε }.
- FIRST(Z): Z → k gives { k }. Z → ε gives ε. Total { k, ε }.
- FOLLOW(S): start symbol → add $. Total { $ }.
- FOLLOW(X): in S → X Y z: after X is Y z → add { y, z }. Total { y, z }.
- FOLLOW(Y): in S → X Y z: after Y is z → add { z }. Total { z }.
- FOLLOW(Z): in Y → y Z: nothing after Z → add FOLLOW(Y). Total { z }.

> **Lose marks by:** FIRST(S) needs z too (X and Y can both be ε).

Answer 3 [5]

WRITE THIS

#	Answer	Reason (one line)
(a)	**False**	A top-down parser loops forever on A → Aα.
(b)	**True**	If A ⇒* ε then ε ∈ FIRST(A). (FOLLOW never has ε.)
(c)	**False**	Two rules in one cell, so not LL(1).
(d)	**True**	The scanner throws them away while making tokens.
(e)	**False**	It only rewrites the rules; the strings stay the same.

Quiz Answer 1 [5]

WRITE THIS

Round 1: in S, `a b c S`, `a b d`, `a b`, `a T` all start with `a`. Take it out once:

```
S → a S' | e
S' → b c S | b d | b | T
```

Round 2: in S', `b c S`, `b d`, `b` all start with `b`. Take it out once:

```
S' → b S'' | T
S'' → c S | d | ε
```

Final DG:

```
S → a S' | e
S' → b S'' | T
S'' → c S | d | ε
T → t
```

Quiz Answer 2 [5]

WRITE THIS

A → A b c | A d | e | f B: α = `b c`, `d`, β = `e`, `f B`.

```
A → e A' | f B A'
A' → b c A' | d A' | ε
```

B → B g | h: α = `g`, β = `h`.

```
B → h B'
B' → g B' | ε
```

Final RRG:

```
A → e A' | f B A'
A' → b c A' | d A' | ε
B → h B'
B' → g B' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

Answer 4 [8]

WRITE THIS

Step 0 · Remove left recursion

L → L ; D | D: α = `; D`, β = `D`.

```
L → D L'
L' → ; D L' | ε
```

Final RRG:

```
L → D L'
L' → ; D L' | ε
D → int id | float id
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

D → int id | float id start with different terminals, so no left factoring is needed.

Numbered rules: 1. L → D L' 2. L' → ; D L' 3. L' → ε 4. D → int id 5. D → float id

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
L	{ int, float }	{ $ }
L'	{ ;, ε }	{ $ }
D	{ int, float }	{ ;, $ }

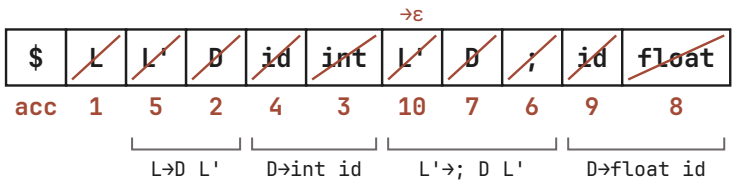
Step 2 · LL(1) parsing table

	;	int	id	float	$
L		L→D L'		L→D L'	
L'	L'→; D L'				L'→ε
D		D→int id		D→float id	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3 · Parse `int id ; float id` (horizontal stack)

Buffer: `int id ; float id $` LA ↑ starts at `int`



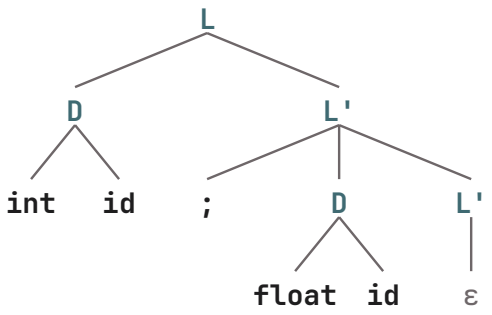
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop L (LA = int): L→D L', push L' D
2. pop D (LA = int): D→int id, push id int
3. match int, pop it, move LA → id
4. match id, pop it, move LA → ;
5. pop L' (LA = ;): L'→; D L', push L' D ;
6. match ;, pop it, move LA → float
7. pop D (LA = float): D→float id, push id float
8. match float, pop it, move LA → id
9. match id, pop it, move LA → $
10. pop L' (LA = $): L'→ε, push nothing

acc. Top = $ and LA = $ → accept

Parse tree



Leaves left to right (skip ε): `int id ; float id` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(L): L → D L' gives { int, float }. Total { int, float }.
- FIRST(L'): L' → ; D L' gives { ; }. L' → ε gives ε. Total { ;, ε }.
- FIRST(D): D → int id gives { int }. D → float id gives { float }. Total { int, float }.
- FOLLOW(L): start symbol → add $. Total { $ }.
- FOLLOW(L'): in L → D L': nothing after L' → add FOLLOW(L). Total { $ }.
- FOLLOW(D): in L → D L': after D is L' → add { ; }; L' can be ε → add FOLLOW(L). in L' → ; D L': after D is L' → add { ; }; L' can be ε → add FOLLOW(L'). Total { ;, $ }.